

Get started

- Overview
- Quickstart
- Models
- Pricing
- SDKs and CLI
- Latest: GPT-5.5
- Prompt guidance

Core concepts

- Text generation
- Code generation
- Images and vision
- Audio and speech
- Structured output
- Function calling
- Responses API
- Using tools

Agents SDK

- Overview
- Quickstart
- Agent definitions
- Models and providers
- Running agents
- Sandbox agents
- Orchestration
- Guardrails
- Results and state
- Integrations and observability
- Evaluate agent workflows
- Voice agents
- Agent Builder

Tools

- Web search
- MCP and Connectors
- Skills
- Shell
- Computer use
- File search and retrieval
- Tool search
- More tools

Run and scale

- Conversation state
- Background mode
- Streaming
- WebSocket mode
- Webhooks
- File inputs
- Context management
- Prompting
- Overview
- Prompt engineering
- Citation formatting
- Migration guide
- Reasoning

Prompt engineering

Enhance results with prompt engineering strategies.

Copy Page

With the OpenAI API, you can use a [large language model](#) to generate text from a prompt, as you might using [ChatGPT](#). Models can generate almost any kind of text response—like code, mathematical equations, structured JSON data, or human-like prose.

Here's a simple example using the [Responses API](#).

Generate text from a simple prompt

javascript

```
1 import OpenAI from "openai";
2 const client = new OpenAI();
3
4 const response = await client.responses.create({
5   model: "gpt-5.5",
6   input: "Write a one-sentence bedtime story about a unicorn."
7 });
8
9 console.log(response.output_text);
```

An array of content generated by the model is in the `output` property of the response. In this simple example, we have just one output which looks like this:

```
1 [
2   {
3     "id": "msg_67b73f697ba4819183a15cc17d011509",
4     "type": "message",
5     "role": "assistant",
6     "content": [
7       {
8         "type": "output_text",
9         "text": "Under the soft glow of the moon, Luna the unicorn danced through fields of twinkling stardust"
10        "annotations": []
11      }
12    ]
13  }
14 ]
```

The `output` array often has more than one item in it! It can contain tool calls, data about reasoning tokens generated by [reasoning models](#), and other items. It is not safe to assume that the model's text output is present at `output[0].content[0].text`.

Some of our [official SDKs](#) include an `output_text` property on model responses for convenience, which aggregates all text outputs from the model into a single string. This may be useful as a shortcut to access text output from the model.

In addition to plain text, you can also have the model return structured data in JSON format - this feature is called [Structured Outputs](#).

Choosing a model

A key choice to make when generating content through the API is which model you want to use - the `model` parameter of the code samples above. [You can find a full listing of available models here](#). Here are a few factors to consider when choosing a model for text generation.

- [Reasoning models](#) generate an internal chain of thought to analyze the input prompt, and excel at understanding complex tasks and multi-step planning. They are also generally slower and more expensive to use than GPT models.
- [GPT models](#) are fast, cost-efficient, and highly intelligent, but benefit from more explicit instructions around how to accomplish tasks.
- [Large and small \(mini or nano\) models](#) offer trade-offs for speed, cost, and intelligence. Large models are more effective at understanding prompts and solving problems across domains, while small models are generally faster and cheaper to use.

When in doubt, `gpt-5.5` offers a strong default for general-purpose text generation and prompt iteration.

Prompt engineering

Prompt engineering is the process of writing effective instructions for a model, such that it consistently generates content that meets your requirements.

Because the content generated from a model is non-deterministic, prompting to get your desired output is a mix of art and science. However, you can apply techniques and best practices to get good results consistently.

Some prompt engineering techniques work with every model, like using message roles. But different model types (like reasoning versus GPT models) might need to be prompted differently to produce the best results. Even different snapshots of models within the same family could produce different results. So as you build more complex applications, we strongly recommend:

- Pinning your production applications to specific [model snapshots](#) (like `gpt-4.1-2025-04-14` for example) to ensure

Responses

- Choosing a model
- Prompt engineering
- Message roles and instruction following
- Reusable prompts
- Message formatting with Markdown and XML
- Few-shot learning
- Include relevant context information
- Prompting current GPT-5 series models
- Prompting reasoning models
- Next steps
- Other resources

Copy Page

- consistent behavior
- Building **evals** that measure the behavior of your prompts so you can monitor prompt performance as you iterate, or when you change and upgrade model versions

Now, let's examine some tools and techniques available to you to construct prompts.

Message roles and instruction following

You can provide instructions to the model with **differing levels of authority** using the `instructions` API parameter or **message roles**.

The `instructions` parameter gives the model high-level instructions on how it should behave while generating a response, including tone, goals, and examples of correct responses. Any instructions provided this way will take priority over a prompt in the `input` parameter.

```
Generate text with instructions                                javascript ↕
1 import OpenAI from "openai";
2 const client = new OpenAI();
3
4 const response = await client.responses.create({
5   model: "gpt-5",
6   reasoning: { effort: "low" },
7   instructions: "Talk like a pirate.",
8   input: "Are semicolons optional in JavaScript?",
9 });
10
11 console.log(response.output_text);
```

The example above is roughly equivalent to using the following input messages in the `input` array:

```
Generate text with messages using different roles              javascript ↕
1 import OpenAI from "openai";
2 const client = new OpenAI();
3
4 const response = await client.responses.create({
5   model: "gpt-5",
6   reasoning: { effort: "low" },
7   input: [
8     {
9       role: "developer",
10      content: "Talk like a pirate."
11     },
12     {
13       role: "user",
14       content: "Are semicolons optional in JavaScript?",
15     },
16   ],
17 });
18
19 console.log(response.output_text);
```

Note that the `instructions` parameter only applies to the current response generation request. If you are **managing conversation state** with the `previous_response_id` parameter, the `instructions` used on previous turns will not be present in the context.

The **OpenAI model spec** describes how our models give different levels of priority to messages with different roles.

developer	user	assistant
<code>developer</code> messages are instructions provided by the application developer, prioritized ahead of <code>user</code> messages.	<code>user</code> messages are instructions provided by an end user, prioritized behind <code>developer</code> messages.	Messages generated by the model have the <code>assistant</code> role.

A multi-turn conversation may consist of several messages of these types, along with other content types provided by both you and the model. Learn more about **managing conversation state here**.

You could think about `developer` and `user` messages like a function and its arguments in a programming language.

- `developer` messages provide the system's rules and business logic, like a function definition.
- `user` messages provide inputs and configuration to which the `developer` message instructions are applied, like arguments to a function.

Reusable prompts

In the OpenAI dashboard, you can develop reusable **prompts** that you can use in API requests, rather than specifying the content of prompts in code. This way, you can more easily build and evaluate your prompts, and deploy improved versions of your prompts without changing your integration code.

Here's how it works:

- Create a reusable prompt** in the **dashboard** with placeholders like `{{customer_name}}`.
- Use the prompt** in your API request with the `prompt` parameter. The prompt parameter object has three properties you can configure:
 - `id` — Unique identifier of your prompt, found in the dashboard
 - `version` — A specific version of your prompt (defaults to the "current" version as specified in the dashboard)

- **variables** — A map of values to substitute in for variables in your prompt. The substitution values can either be strings, or other Response input message types like `input_image` or `input_file`. [See the full API reference.](#)

String variables Variables with file input

Generate text with a prompt template

javascript ↕

```
1 import OpenAI from "openai";
2 const client = new OpenAI();
3
4 const response = await client.responses.create({
5   model: "gpt-5",
6   prompt: {
7     id: "pmpmt_abc123",
8     version: "2",
9     variables: {
10      customer_name: "Jane Doe",
11      product: "48oz juice box"
12    }
13  }
14 });
15
16 console.log(response.output_text);
```

Message formatting with Markdown and XML

When writing `developer` and `user` messages, you can help the model understand logical boundaries of your prompt and context data using a combination of [Markdown](#) formatting and [XML tags](#).

Markdown headers and lists can be helpful to mark distinct sections of a prompt, and to communicate hierarchy to the model. They can also potentially make your prompts more readable during development. XML tags can help delineate where one piece of content (like a supporting document used for reference) begins and ends. XML attributes can also be used to define metadata about content in the prompt that can be referenced by your instructions.

In general, a developer message will contain the following sections, usually in this order (though the exact optimal content and order may vary by which model you are using):

- **Identity:** Describe the purpose, communication style, and high-level goals of the assistant.
- **Instructions:** Provide guidance to the model on how to generate the response you want. What rules should it follow? What should the model do, and what should the model never do? This section could contain many subsections as relevant for your use case, like how the model should [call custom functions](#).
- **Examples:** Provide examples of possible inputs, along with the desired output from the model.
- **Context:** Give the model any additional information it might need to generate a response, like private/proprietary data outside its training data, or any other data you know will be particularly relevant. This content is usually best positioned near the end of your prompt, as you may include different context for different generation requests.

Below is an example of using Markdown and XML tags to construct a `developer` message with distinct sections and supporting examples.

Example prompt API request

A developer message for code generation

↕

```
1 # Identity
2
3 You are coding assistant that helps enforce the use of snake case
4 variables in JavaScript code, and writing code that will run in
5 Internet Explorer version 6.
6
7 # Instructions
8
9 * When defining variables, use snake case names (e.g. my_variable)
10 instead of camel case names (e.g. myVariable).
11 * To support old browsers, declare variables using the older
12 "var" keyword.
13 * Do not give responses with Markdown formatting, just return
14 the code as requested.
15
16 # Examples
17
18 <user_query>
19 How do I declare a string variable for a first name?
20 </user_query>
21
22 <assistant_response>
23 var first_name = "Anna";
24 </assistant_response>
```

Save on cost and latency with prompt caching

When constructing a message, you should try and keep content that you expect to use over and over in your API requests at the beginning of your prompt, **and** among the first API parameters you pass in the JSON request body to [Chat Completions](#) or [Responses](#). This enables you to maximize cost and latency savings from [prompt caching](#).

Few-shot learning

Few-shot learning lets you steer a large language model toward a new task by including a handful of input/output examples in the prompt, rather than [fine-tuning](#) the model. The model implicitly "picks up" the pattern from those examples and applies it to a prompt. When providing examples, try to show a diverse range of possible inputs with the desired outputs.

Typically, you will provide examples as part of a `developer` message in your API request. Here's an example `developer` message containing examples that show a model how to classify positive or negative customer service reviews.

```
# Identity

You are a helpful assistant that labels short product reviews as
Positive, Negative, or Neutral.

# Instructions

* Only output a single word in your response with no additional formatting
  or commentary.
* Your response should only be one of the words "Positive", "Negative", or
  "Neutral" depending on the sentiment of the product review you are given.

# Examples

<product_review id="example-1">
I absolutely love this headphones — sound quality is amazing!
</product_review>

<assistant_response id="example-1">
Positive
</assistant_response>

<product_review id="example-2">
Battery life is okay, but the ear pads feel cheap.
</product_review>

<assistant_response id="example-2">
Neutral
</assistant_response>

<product_review id="example-3">
Terrible customer service, I'll never buy from them again.
</product_review>

<assistant_response id="example-3">
Negative
</assistant_response>
```

Include relevant context information

It is often useful to include additional context information the model can use to generate a response within the prompt you give the model. There are a few common reasons why you might do this:

- To give the model access to proprietary data, or any other data outside the data set the model was trained on.
- To constrain the model's response to a specific set of resources that you have determined will be most beneficial.

The technique of adding additional relevant context to the model generation request is sometimes called **retrieval-augmented generation (RAG)**. You can add additional context to the prompt in many different ways, from querying a vector database and including the text you get back into a prompt, or by using OpenAI's built-in [file search tool](#) to generate content based on uploaded documents.

Planning for the context window

Models can only handle so much data within the context they consider during a generation request. This memory limit is called a **context window**, which is defined in terms of [tokens](#) (chunks of data you pass in, from text to images).

Models have different context window sizes from the low 100k range up to one million tokens for newer GPT-4.1 models. [Refer to the model docs](#) for specific context window sizes per model.

Prompting current GPT-5 series models

GPT models like `gpt-5.5` benefit from precise instructions that explicitly provide the logic and data required to complete the task in the prompt. To get the most out of the latest GPT-5 series model, start with the current prompting guide.



GPT-5.5 prompting guide

Get the most out of prompting the latest GPT-5 series model with current guidance, practical examples, and migration notes.

Prompting best practices for the latest GPT-5 series model

For the full current treatment, use the [prompt guidance](#) guide. The practical reminders below still apply.

> Coding

> Front-end engineering

> Agentic tasks

Prompting reasoning models

There are some differences to consider when prompting a [reasoning model](#) versus prompting a GPT model. Generally speaking, reasoning models will provide better results on tasks with only high-level guidance. This differs from GPT models, which benefit from very precise instructions.

You could think about the difference between reasoning and GPT models like this.

- A reasoning model is like a senior co-worker. You can give them a goal to achieve and trust them to work out the

details.

- A GPT model is like a junior coworker. They'll perform best with explicit instructions to create a specific output.

For more information on best practices when using reasoning models, [refer to this guide](#).

Next steps

Now that you know the basics of text inputs and outputs, you might want to check out one of these resources next.

-  **Build a prompt in the Playground**
Use the Playground to develop and iterate on prompts.
-  **Generate JSON data with Structured Outputs**
Ensure JSON data emitted from a model conforms to a JSON schema.
-  **Full API reference**
Check out all the options for text generation in the API reference.

Other resources

For more inspiration, visit the [OpenAI Cookbook](#), which contains example code and also links to third-party resources such as:

- [Prompting libraries & tools](#)
- [Prompting guides](#)
- [Video courses](#)
- [Papers on advanced prompting to improve reasoning](#)